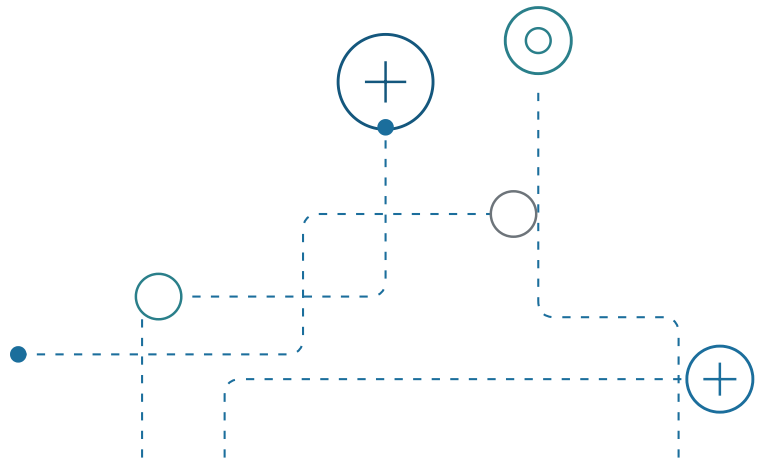


AI 에이전트 프로토콜 2026

MCP-A2A 오케스트레이션 표준 지형

bookforge



Contents

에이전트 표준 지형 개관

세 개의 축, 세 개의 질문

이 책이 다루는 순서

1

3

4

MCP 아키텍처와 원리

호스트·클라이언트·서버, 세 개의 역할

서버가 노출하는 세 가지 원시

발견에서 응답까지, 호출의 흐름

6

7

8

9

A2A와 에이전트 간 협업

Agent Card, 능력을 게시하는 방법

태스크 생명주기 — 위임에서 반환까지

11

12

13

멀티 에이전트 오케스트레이션

운영이 드러내는 현실

16

19

신뢰와 권한의 경계 설계

도구 노출 지점: MCP 서버의 과잉 권한

협업 지점: A2A에서 상대를 검증하는 문제

지휘 지점: 위임 체인 전체의 최소 권한

21

22

22

23

표준 경쟁과 상호운용성

경쟁하는 제안들이 수렴해 가는 경로

지금 조직이 취할 수 있는 실행 원칙

다섯 장을 관통한 흐름

26

27

28

28

01

에이전트 표준 지형 개관

단일 도구를 부르던 에이전트가 서로 통신하는 시스템으로 넘어가면서, 그 연결을 지탱하는 프로토콜 계층이 갑자기 핵심 화두로 떠올랐다. 이 장은 통합 비용이 급셈으로 불어나는 구조적 이유를 먼저 짚고, MCPA2A· 오케스트레이션이라는 세 층위가 각각 어떤 질문에 답하는 표준인지 구분해 이후 다섯 장이 걸어갈 지도를 그린다.

몇 해 전까지 AI 에이전트가 하는 일은 단순했다. 하나의 언어모델이 질문을 받고, 필요하면 미리 정해진 도구 하나를 불러 답을 완성했다. 검색 API를 호출하거나 계산기를 두드리는 수준이었고, 이 관계는 언제나 모델 하나가 도구 하나를 상대하는 일대일 구조였다. 그런데 에이전트가 처리하는 작업이 복잡해지면서 이 구조는 더는 충분하지 않게 되었다. 하나의 에이전트가 여러 도구를 넘나들고, 여러 에이전트가 서로 결과를 주고받으며, 어떤 에이전트는 다른 에이전트에게 작업 전체를 위임한다. 도구를 부르던 존재가 서로 통신하는 시스템으로 넘어간 것이다.

이 전환이 갑자기 '표준 규격'이라는 말을 화두로 밀어올린 이유는 비교적 단순하다. 통합의 개수가 곱셈으로 불어나기 때문이다. 여러 에이전트가 여러 도구-데이터 소스에 각자 다른 방식으로 연결되면, 필요한 커넥터의 수는 참여자 수의 곱에 가까워진다. 도구가 바뀌거나 에이전트 프레임워크가 바뀔 때마다 이 커넥터를 하나씩 다시 짜야 한다면, 연결을 유지하는 비용이 연결이 만드는 가치를 갉아먹는다.

N×M 파편화 문제

에이전트마다, 도구마다 각자의 방식으로 통합을 짜면 연결의 수가 참여자 수의 곱으로 늘어난다. 프로토콜은 이 곱셈 구조를 공통 인터페이스 하나로 접어, 참여자가 늘어도 연결 방식 자체는 하나로 유지되게 한다.

프로토콜이 하는 일은 바로 이 곱셈을 덧셈으로 바꾸는 것이다. 각 도구가, 각 에이전트가 정해진 하나의 인터페이스만 지키면, 나머지 참여자는 그 인터페이스만 보고 연결할 수 있다. 새로운 도구가 하나 늘어도, 새로운 에이전트가 하나 늘어도 기존 연결을 다시 손대지 않아도 된다는 뜻이다. 무엇을 표준화하고 무엇을 벤더의 재량으로 남기느냐는 다음 장대에서 하나씩 다루지만, 이 장에서는 먼저 표준화가 필요한 지점 자체를 층위별로 구분해 두려 한다. 층위를 먼저 나누지 않으면, 도구 연결의 문제와 에이전트 간 통신의 문제와 다중 에이전트 조율의 문제를 뭉뚱그려 다루게 되고, 그러면 어떤 프로토콜이 무엇을 해결해 주는지도 흐려진다.

1.1 세 개의 축, 세 개의 질문

이 책은 에이전트 시스템을 지탱하는 프로토콜 계층을 세 개의 축으로 나눠 다룬다. 세 축은 같은 문제를 다루지 않는다. 각자 다른 질문에 답한다.

첫째는 **MCP**(Model Context Protocol)다. "에이전트가 도구와 데이터에 어떻게 접근하는가"라는 질문에 답한다. 모델과 그 바깥 세계를 잇는다는 점에서 수직적인 연결이다.

둘째는 **A2A**(Agent-to-Agent)다. "서로 다른 조직·프레임워크에서 만든 에이전트가 어떻게 서로에게 일을 맡기는가"에 답한다. 에이전트와 에이전트를 잇는다는 점에서 수평적인 연결이다.

셋째는 **오케스트레이션**이다. 앞의 두 프로토콜이 연결 방식을 표준화해도, 여러 에이전트를 하나의 목표로 묶어 조율하는 설계는 여전히 개발자의 몫으로 남는다. "누가 무엇을 언제 하는가"를 정하는 상위 층위다.

세 축을 나란히 놓고 보면 이 층위 구분이 더 뚜렷해진다. 아래에는 에이전트를 도구에 잇는 배관이 있고, 그 옆에는 에이전트끼리 잇는 배관이 있으며, 그 위에는 그 배관들을 지휘하는 설계가 있다. 배관이 아무리 튼튼해도 지휘가 없으면 각 연결은 흩어진 채로 남고, 지휘가 아무리 정교해도 배관이 없으면 지휘할 대상 자체가 연결되지 않는다.

이 구분이 실무에서 값을 하는 순간은 시스템이 말썽을 일으킬 때다. 도구 호출이 실패하는 문제는 MCP 층위에서, 위임한 작업이 엉뚱한 결과로 돌아오는 문제는 A2A 층위에서, 전체 흐름이 꼬이는 문제는 오케스트레이션 층위에서 원인을 찾아야 한다. 층위를 혼동하면 배관의 문제를 지휘 로직으로 고치려 들거나 지휘의 문제를 배관 탓으로 돌리게 되고, 고칠 곳을 찾는 시간만 하염없이 길어진다. 진단의 첫걸음은 언제나 문제의 속한 층위를 지목하는 일이다.

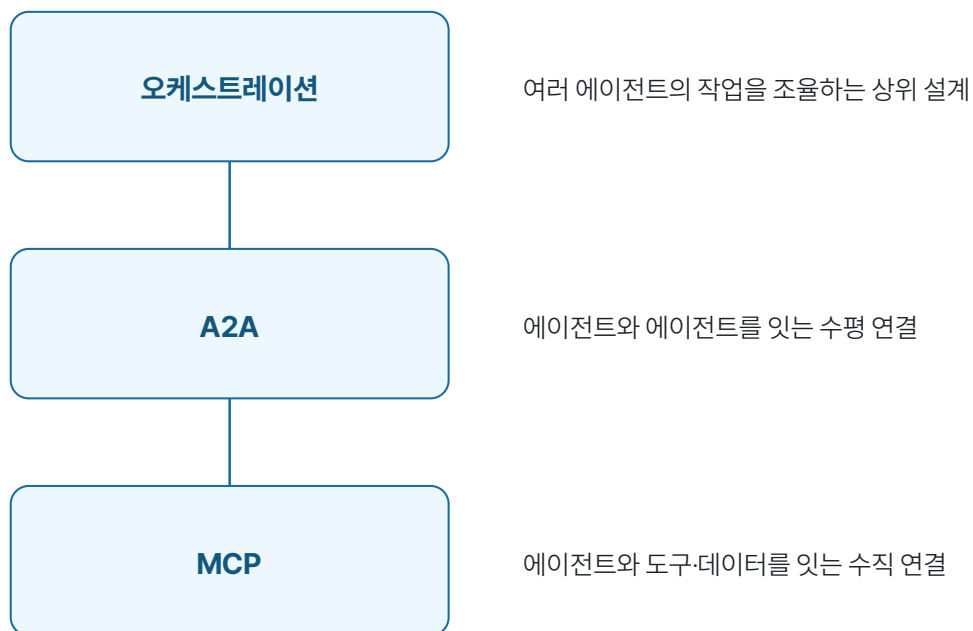


그림 1-1 MCP·A2A·오케스트레이션 세 축이 각각 담당하는 연결과 조율의 층위 · 출처: 본문 정리

1.2 이 책이 다루는 순서

이후 장은 이 세 축을 순서대로 파고든다. 2장은 MCP의 호스트-클라이언트-서버 구조와 도구·리소스·프롬프트라는 세 원시 타입을 다룬다. 3장은 A2A가 규정하는 에이전트 간 위임의 절차를 살핀다. 4장은 여러 에이전트를 하나의 작업으로 묶는 조율 패턴을 정리한다. 5장은 이 연결들이 늘어날수록 함께 커지는 권한과 신뢰의 문제를 짚고, 6장은 아직 하나로 수렴하지 않은 표준 경쟁의 구도에서 조직이 지금 취할 수 있는 태도로 이 책을 맺는다.

읽는 법

세 축의 관계가 헷갈리면 질문을 먼저 떠올리면 된다. "이 에이전트가 무엇을 쓸 수 있는가"는 MCP의 몫이고, "이 에이전트가 누구에게 일을 맡길 수 있는가"는 A2A의 몫이며, "누가 언제 무엇을 하는가"는 오케스트레이션의 몫이다.

표준이 자리잡는 과정은 대개 어수선하다. 여러 제안이 동시에 등장하고, 비슷한 문제를 다르게 풀려는 시도들이 한동안 나란히 존재한다. 그럼에도 이 장에서 그은 세 축의 구분 자체는 유지될 가능성이 크다. 도구 연결과 에이전트 간 통신과 다중 에이전트 조율은 성격이 다른 문제이고, 문제가 다르면 그 문제에 답하는 표준도 계속 따로 존재할 이유가 있기 때문이다. 어느 프로토콜이 최종 승자가 되느냐보다 중요한 것은, 지금 겪고 있는 문제가 이 세 층위 가운데 어디에 속하는지를 먼저 가려내는 일이다. 이어지는 장들은 이 세 층위를 하나씩 열어 그 안의 구체적인 구조를 들여다본다.

02

MCP 아키텍처와 원리

에이전트가 외부 도구·데이터에 안전하게 접근하도록 만든 표준, MCP(Model Context Protocol)는 정확히 무엇을 표준화하고 무엇을 벤더 재량에 남겨두는가. 호스트·클라이언트·서버라는 세 역할의 분담, 서버가 노출하는 세 가지 원시, 그리고 발견에서 응답까지 이어지는 호출의 흐름을 사양의 눈높이에서 차례로 해부한다.

에이전트가 외부 도구나 데이터베이스를 다루려면 그 사이를 잇는 연결 코드가 필요하다. 문제는 애플리케이션과 도구의 조합마다 이 연결을 새로 짜야 한다는 데 있었다. 검색 도구 하나를 세 개의 에이전트 애플리케이션에서 쓰려면 세 벌의 커스텀 통합이 필요했고, 도구가 늘고 애플리케이션이 늘수록 짜야 할 통합의 수는 곱셈으로 불어났다. 통합 하나 하나는 인증 방식도, 결과를 돌려주는 형식도 제각각이었기 때문에, 도구 제공자는 애플리케이션마다 다른 문서를 만들어야 했고 애플리케이션 개발자는 도구마다 다른 클라이언트 코드를 유지해야 했다. MCP(Model Context Protocol)는 이 반복을 없애기 위해 등장했다 — 도구를 "어떻게 부르는가"를 애플리케이션마다 다시 정의하지 않도록, 부르는 방식 자체를 표준 인터페이스로 굳힌 것이다. 도구 제공자는 이 인터페이스에 맞춰 서버 하나만 만들면 되고, 애플리케이션 개발자는 그 인터페이스를 아는 클라이언트 하나만 구현하면 된다.

2.1 호스트·클라이언트·서버, 세 개의 역할

MCP는 세 역할로 연결을 나눈다. **호스트**는 사용자가 실제로 쓰는 에이전트 애플리케이션이다. 호스트 안에는 **클라이언트**가 있어 하나의 외부 연결을 전담한다. 클라이언트가 실제로 접속하는 상대가 **서버**로, 특정 도구나 데이터 소스를 감싸 표준 방식으로 노출하는 쪽이다. 호스트 하나가 여러 클라이언트를 두고, 각 클라이언트가 서로 다른 서버 하나씩과 1:1로 연결되는 구조다. 이 분리 덕분에 서버를 만드는 쪽은 어떤 호스트가 자신을 부를지 몰라도 되고, 호스트를 만드는 쪽은 서버 내부가 어떻게 구현되었는지 몰라도 된다.

구조의 핵심

호스트-클라이언트-서버 분리는 "누가 무엇을 아는가"를 최소화한다. 서버는 호스트를 모르고, 호스트는 서버 내부를 모른 채 표준 인터페이스만 신뢰한다.

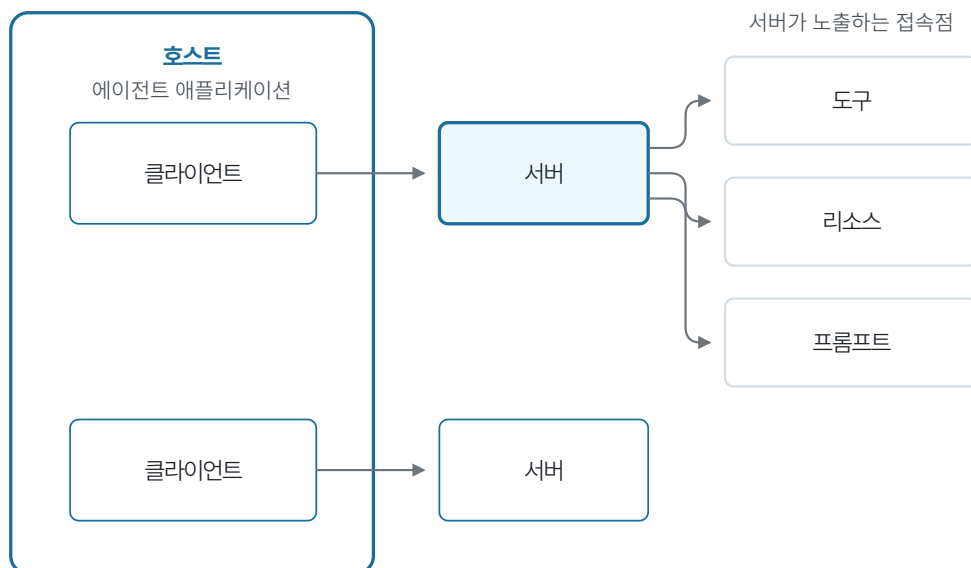


그림 2-1 MCP 호스트-클라이언트-서버 3계층 구조와 도구-리소스-프롬프트 접속점 · 출처: MCP 사양 구조 재구성

2.2 서버가 노출하는 세 가지 원시

서버는 자신의 기능을 세 가지 원시 타입으로 표현한다. 도구(tools)는 에이전트가 호출해 부수 효과를 일으키거나 결과를 받는 실행 단위다. 파일을 쓰거나, 외부 API를 두드리거나, 계산을 수행하는 동작이 여기 속한다. 리소스(resources)는 파일이나 레코드처럼 에이전트가 맥락으로 읽어들이는 데이터다. 실행이 아니라 참조를 위한 원시라는 점에서 도구와 구분된다. 프롬프트(prompts)는 자주 쓰는 지시문을 서버 쪽에서 미리 정의해 재사용하게 만든 템플릿이다. 세 원시는 역할이 뚜렷이 갈리지만, 모든 서버가 세 가지를 다 갖추어야 하는 것은 아니다 — 도구만 노출하는 서버도, 리소스만 노출하는 서버도 흔하다.

이 편차 때문에 클라이언트는 연결 초기에 서버가 이 세 원시 중 무엇을 지원하는지 확인하는 절차를 거치는데, 이를 능력 협상(capability negotiation)이라 부른다. 협상을 거치면 클라이언트는 서버가 실제로 제공하는 것만 사용자에게 제시할 수 있어, 지원하지 않는 기능을 호출해 실패하는 상황을 미리 막는다.

2.3 발견에서 응답까지, 호출의 흐름

능력 협상이 끝나면 클라이언트는 서버가 제공하는 도구 목록을 요청해 무엇을 부를 수 있는지 파악한다. 이후 에이전트가 특정 작업을 위해 도구 하나를 선택하면, 클라이언트는 해당 도구 이름과 인자를 서버에 전달하고 서버는 도구를 실행해 결과를 되돌려준다. 이 발견-호출-응답의 순서는 매 요청마다 반복되는 것이 아니라, 한 번 확인한 목록을 세션 동안 재사용하는 방식으로 오버헤드를 줄인다.

표 2-1. MCP 전송 방식 비교

방식	연결 대상	성격
로컬(stdio)	같은 기기의 프로세스	표준입출력을 통한 직접 파이프
원격(HTTP)	네트워크 너머 서버	요청-응답 기반 접속

자료: 저자 정리

로컬 전송은 표준입출력으로 프로세스를 직접 연결해 지연이 거의 없고, 원격 전송은 네트워크 너머의 서버에 접속해 여러 사용자가 하나의 서버를 공유할 수 있게 한다. 개인 환경에서 파일 시스템처럼 로컬 자원을 다루는 서버는 stdio 쪽이 자연스럽고, 여러 사용자가 함께 접근해야 하는 조직 내부 데이터나 SaaS 연동은 원격 쪽이 맞는

경우가 많다. 어느 쪽을 쓰든 호출 흐름 자체는 동일하며, 여러 벤더가 이 두 방식을 함께 지원하는 서버를 만들고 있다.

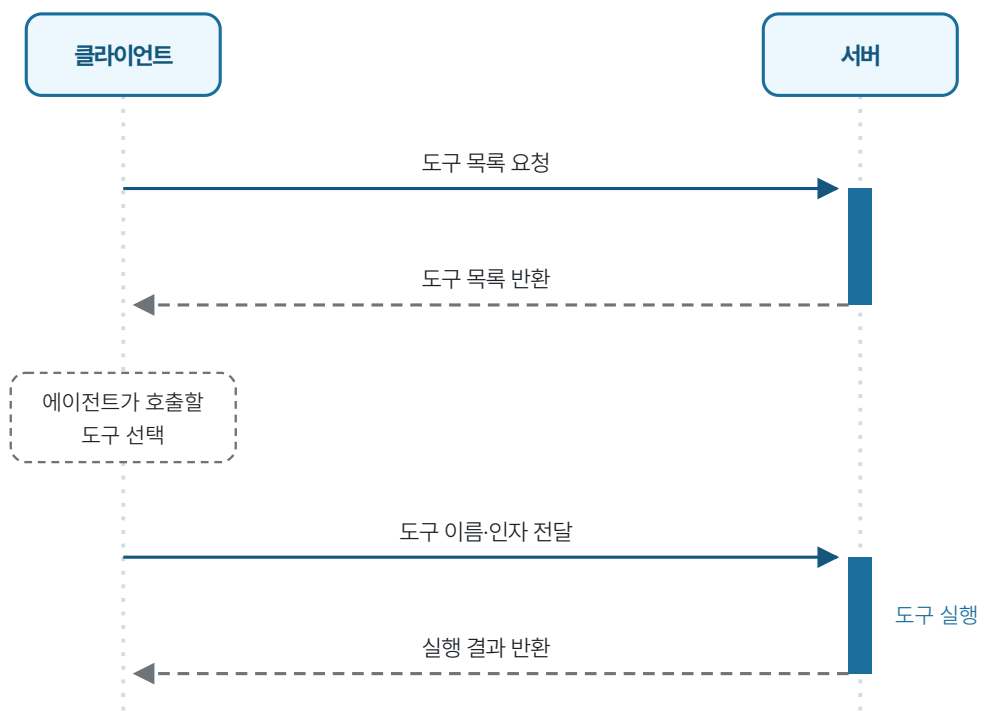


그림 2-2 클라이언트가 서버의 도구를 발견하고 호출해 결과를 받는 요청-응답 흐름 · 출처: MCP 사양
흐름 재구성

표준화의 경계

MCP가 못 박는 것은 도구·리소스·프롬프트를 주고받는 메시지 형식과 절차이지, 서버 내부의 구현 방식이 아니다. 어떤 언어로, 어떤 데이터베이스로 서버를 짜는지는 전적으로 개발자 재량으로 남는다.

03

A2A와 에이전트 간 협업

서로 다른 조직·프레임워크에서 만든 에이전트들이 서로를 신뢰하고 작업을 위임할 수 있으려면 어떤 통신 규약이 필요한가. 능력을 게시하는 Agent Card에서 출발해, 태스크가 생성되고 진행 상태를 오가다 결과물로 반환되기까지의 생명주기를 따라가며 A2A가 표준화해 둔 협업의 최소 단위를 확인한다.

앞 장에서 다룬 MCP는 에이전트와 도구를 잇는 연결이다. 에이전트가 파일 시스템을 읽거나 검색 API를 호출할 때, MCP는 그 호출이 어떤 형식으로 오가야 하는지를 표준화한다. 이 연결은 방향이 분명하다 — 위에는 판단하는 에이전트가 있고 아래에는 실행되는 도구가 있는, 수직 관계다.

A2A(Agent-to-Agent)가 다루는 문제는 층위 자체가 다르다. 한 조직이 만든 에이전트가 다른 조직, 다른 프레임워크, 심지어 다른 회사가 만든 에이전트에게 작업 일부를 넘겨야 하는 상황을 생각해 보자. 양쪽 모두 스스로 판단하고 행동하는 주체이므로, 이 연결에는 위아래가 없다. 여행 일정을 조율하는 에이전트가 항공권 예약을 전달하는 별도 에이전트에게 구간을 통째로 맡기는 식이다. MCP가 에이전트 내부에서 바깥의 도구로 뻗는 수직선이라면, A2A는 에이전트와 에이전트 사이를 잇는 수평선이다.

이 구분이 중요한 이유는 두 프로토콜이 풀어야 하는 문제의 성격이 다르기 때문이다. 도구는 스스로 판단하지 않는다 — 정해진 입력을 주면 정해진 출력을 돌려주는 함수에 가깝다. 반면 위임받은 에이전트는 요청을 해석하고, 필요하면 자신의 하위 도구를 부르고, 상황에 따라 다른 경로로 작업을 완수할 수도 있다. 그래서 A2A는 단순한 호출-응답이 아니라, 자율적으로 움직이는 상대와 신뢰를 주고받는 절차를 필요로 한다. 하나의 시스템 안에서 이 두 축은 종종 함께 쓰인다 — 최상위 에이전트가 A2A로 하위 에이전트에게 작업을 나누고, 각 하위 에이전트는 다시 MCP로 자신의 도구를 불러 실제 작업을 수행하는 구조다.

3.1 Agent Card, 능력을 게시하는 방법

수평 연결이 성립하려면 먼저 상대가 무엇을 할 수 있는지 알아야 한다. A2A는 이를 위해 Agent Card라는 개념을 둔다. 각 에이전트가 자신이 처리할 수 있는 작업의 종류, 입출력 형식, 접속 지점을 기술한 명세를 스스로 게시하는 방식이다. 클라이언트 역할을 하는 에이전트는 작업을 넘기기 전에 이 카드를 조회해, 상대가 요청을 처리할

능력이 있는지부터 확인한다. 사람이 이력서를 보고 협업 여부를 판단하는 것과 비슷한 절차를, 에이전트 사이에서 규격화한 셈이다.

이 카드는 단순한 자기소개를 넘어, 협업이 시작되기 전에 서로의 범위를 명확히 긋는 장치이기도 하다. 클라이언트는 카드를 보고 이 에이전트가 다룰 수 없는 작업까지 떠맡기지 않도록 걸러낼 수 있고, 반대로 위임받는 에이전트는 자신이 공개한 범위 밖의 요청을 정당하게 거절할 근거를 갖는다. 두 에이전트가 실제로 대화를 시작하기 전에 이미 상당한 조율이 카드 한 장으로 끝나는 셈이다.

3.2 태스크 생명주기 — 위임에서 반환까지

능력이 확인되면 실제 협업은 정해진 순서를 따라 진행된다. 먼저 클라이언트 에이전트가 태스크를 생성해 상대방에게 위임한다. 위임받은 에이전트는 이를 접수하고 처리에 들어가는데, 작업이 오래 걸리는 경우 클라이언트는 완료를 기다리는 대신 중간중간 상태를 조회할 수 있다. 진행 중, 입력 대기, 완료 같은 상태값이 이 조회에 응답한다. 처리가 끝나면 위임받은 에이전트는 결과물을 아티팩트 형태로 반환하고, 클라이언트는 이를 받아 다음 단계로 넘긴다. 위임과 조회, 반환이라는 세 동작이 하나의 태스크 안에서 순환하는 구조다.

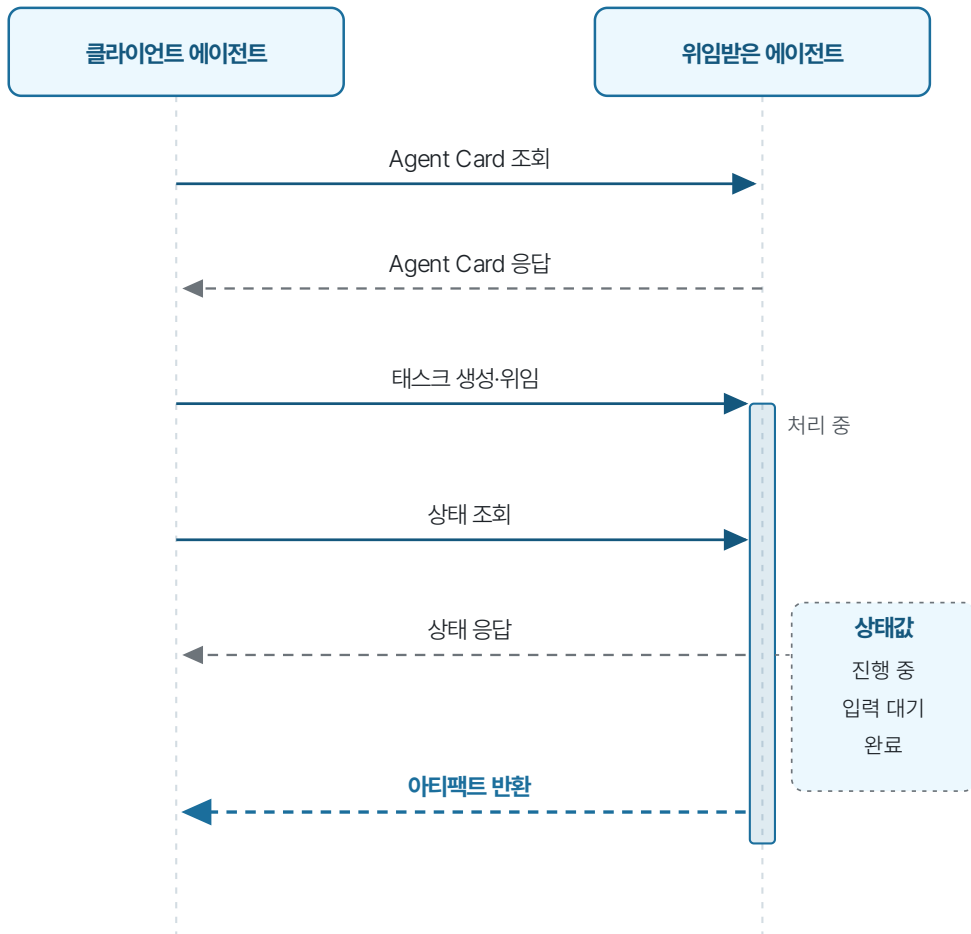


그림 3-1 클라이언트 에이전트가 Agent Card를 조회하고 태스크를 위임해 결과를 받는 생명주기 · 출처: 본문
태스크 생명주기 정리

불투명한 협업

A2A 설계의 핵심은 상대 에이전트의 내부를 몰라도 된다는 데 있다. 어떤 모델을 쓰는지, 어떤 프레임워크로 구현됐는지, 내부 추론 과정이 몇 단계인지는 협업의 조건이 아니다. Agent Card가 약속한 입출력과 태스크 생명주기만 지키면, 내부는 상대방에게 불투명한 채로 남아도 된다.

이 '불투명한 협업(opaque collaboration)' 철학은 실무적인 이유에서 나온다. 협업하는 모든 에이전트가 같은 모델이나 같은 회사의 제품일 이유는 없다. 오히려 서로 다른 조직이 각자 강점을 가진 에이전트를 독립적으로 발전시키면서도 안전하게 맞물릴 수 있어야, 생태계 전체가 한 벤더에 종속되지 않는다. 내부 구현을 감추고 인터페이스만 드러내는 것은 소프트웨어 설계에서 익숙한 원칙이지만, A2A는 이를 에이전트라는 자율적 행위자 사이의 관계에 그대로 옮겨 적용한다.

표준의 현재 위치

A2A 방식의 프로토콜을 둘러싸고 여러 진영이 각자의 제안을 내고 참여하는 단계가 이어지고 있다. 아직 하나의 규격으로 완전히 수렴했다고 보기는 이르며, 조직마다 채택 속도와 방식에도 차이가 있다. 다음 장에서 다룰 오케스트레이션은 이런 수평 연결 위에서 여러 에이전트를 실제로 지휘하는 상위 설계를 다룬다.

결국 A2A가 답하는 질문은 하나로 요약된다. 서로 다른 곳에서 만들어진 에이전트들이 상대의 내부를 몰라도 신뢰하고 일을 맡길 수 있으려면, 무엇을 미리 약속해 두어야 하는가. Agent Card로 능력을 게시하고, 정해진 생명주기로 위임과 반환을 주고받는 것이 그 약속의 최소 단위다.

04

멀티 에이전트 오케스트레이션

개별 프로토콜이 연결을 표준화해도, 여러 에이전트를 하나의 목표로 조율하는 상위 설계는 여전히 개발자의 몫으로 남는다. 이 장은 감독자-워커 구조를 중심으로 작업을 나누고 배정하고 취합하는 조율 패턴을 정리하고, 실제 운영에서 비용과 지연과 실패 전파가 그 설계를 어떻게 되돌려 세우는지까지 살핀다.

MCP가 에이전트와 도구를 잇고 A2A가 에이전트와 에이전트를 잇더라도, 그 연결들을 모아 하나의 목표로 움직이게 만드는 일은 여전히 남는다. 프로토콜이 배관이라면 오케스트레이션은 지휘다. 배관이 물을 어디로도 흘려보낼 수 있게 만드는 표준 규격이라면, 지휘는 지금 이 파이프로 무엇을 언제 흘릴지 정하는 설계도다. 이 층위는 아직 프로토콜만큼 표준화되지 않았고, 대체로 개발자가 애플리케이션 코드 안에서 직접 짜야 하는 몫으로 남아 있다.

두 층위의 관계

MCP·A2A는 "어떻게 연결하는가"에 답하고, 오케스트레이션은 "무엇을 누구에게, 어떤 순서로 맡기는가"에 답한다. 프로토콜이 표준화된 이후에도 조율 설계는 여전히 매 프로젝트마다 새로 결정해야 한다.

조율 패턴은 크게 세 갈래로 나뉜다. 첫째는 감독자-워커 패턴이다. 하나의 감독자 에이전트가 전체 목표를 받아 하위 작업으로 분해하고, 각 작업을 전문화된 워커 에이전트에게 위임한 뒤 돌아온 결과를 취합해 다음 지시를 내리거나 최종 답을 구성한다. 감독자는 워커의 내부 로직을 몰라도 되고, 워커는 다른 워커의 존재조차 몰라도 된다 — 조율의 지식은 감독자 한 곳에 모인다.

감독자-워커 패턴은 아래 흐름으로 요약된다. 감독자가 목표를 분해해 워커별 작업을 배정하고, 워커가 각자의 몫을 병렬로 처리한 뒤 결과를 반환하면, 감독자가 이를 취합해 다음 라운드를 열거나 종료를 판단한다.

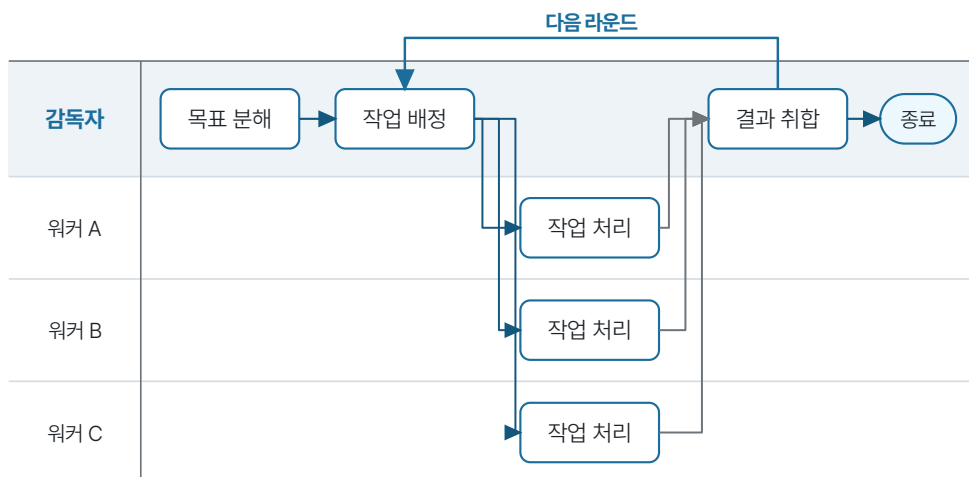


그림 4-1 감독자 에이전트가 작업을 분해해 워커 에이전트들에 위임하고, 각 워커의 결과를 취합해 다음 지시를 내리는 핸드오프 흐름 · 출처: 본문 절차 정리

이 패턴이 잘 맞는 경우는 하위 작업들이 서로 독립적이어서 병렬로 처리해도 순서가 결과를 바꾸지 않을 때다. 반대로 워커 사이에 순서 의존성이 강하면, 감독자가 매번 두 워커의 결과를 맞대어 조정해야 하므로 지휘 비용이 커진다. 감독자 로직을 얼마나 정교하게 짤지 — 단순히 작업을 나눠 던지고 결과를 이어 붙일지, 아니면 중간 결과를 보고 다음 워커 배정을 동적으로 바꿀지 — 가 이 패턴의 설계 폭을 결정한다.

둘째는 파이프라인 패턴이다. 여러 에이전트가 고정된 순서로 늘어서고, 앞 단계의 출력이 그대로 다음 단계의 입력이 된다. 검색 에이전트가 자료를 모으면 요약 에이전트가 정리하고, 검증 에이전트가 사실관계를 확인하는 식이다. 감독자-워커와 달리 중앙의 조율자가 없고 흐름 자체가 순서를 강제하므로 설계는 단순하지만, 중간 단계 하나가 막히면 뒤 단계 전체가 멈추는 취약점을 안는다. 각 단계가 이전 단계의 출력 형식을 그대로 신뢰해야 하므로, 단계 사이의 인터페이스를 얼마나 엄격하게 정의하느냐가 파이프라인 전체의 안정성을 좌우한다.

셋째는 블랙보드 패턴이다. 여러 에이전트가 공유된 상태 저장소(블랙보드)를 함께 들여다보며, 자신이 기여할 수 있는 시점이 오면 스스로 판단해 값을 채운다. 감독자의 명시적 지시도, 파이프라인의 고정 순서도 없다 — 참여자는 느슨하게 결합되어 있고, 언제 누가 관여할지는 상태의 변화가 결정한다. 참여자 구성이 유동적이거나 작업 순

서를 미리 정하기 어려운 문제에 적합하지만, 그만큼 전체 흐름을 추적하고 디버깅하기는 어려워진다. 어느 참여자가 마지막으로 블랙보드를 갱신했는지, 그 갱신이 다른 참여자의 다음 행동을 어떻게 촉발했는지를 되짚기가 세 패턴 중 가장 까다롭다. 세 패턴은 서로 배타적이지 않다. 실무에서는 감독자가 하위 작업 하나를 다시 작은 파이프라인으로 구성하거나, 파이프라인의 한 단계 안에서 여러 워커가 블랙보드를 통해 협업하는 식으로 중첩해 쓰는 경우가 흔하다. 패턴을 고르는 기준은 유행이 아니라 작업 사이의 의존 관계다 — 독립적으로 나뉘면 감독자-워커, 순서가 고정되어 있으면 파이프라인, 참여자와 순서 모두 유동적이면 블랙보드 쪽으로 무게가 실린다.

4.1 운영이 드러내는 현실

패턴을 정했다고 조율이 끝나는 것은 아니다. 실제 운영에서는 실패 처리가 설계의 절반을 차지한다. 워커 하나가 응답하지 않으면 감독자는 얼마나 기다릴지(타임아웃), 실패를 몇 번까지 다시 시도할지(재시도), 그래도 안 되면 그 작업을 건너뛴지 전체를 중단할지를 미리 정해 두어야 한다. 파이프라인에서는 한 단계의 실패가 뒤 단계로 그대로 전파되므로 단계마다 검증 지점을 두는 편이 안전하고, 블랙보드에서는 한 참여자가 잘못된 값을 써넣었을 때 이를 걸러낼 별도의 감시 로직이 필요하다.

재시도가 만드는 함정

재시도 횟수를 넉넉히 잡으면 일시적 오류는 흡수되지만, 워커 자체가 계속 잘못된 결과를 내는 상황에서는 같은 실패를 여러 번 반복하며 시간과 비용만 소모한다. 재시도 상한과 함께 "몇 번 실패하면 사람에게 넘긴다"는 탈출 조건을 반드시 짚지어야 한다.

실패 처리 설계 점검

타임아웃 — 워커의 응답을 얼마나 기다릴지 미리 정한다. 재시도 상한 — 몇 번까지 다시 시도할지, 그래도 안 되면 사람에게 넘길지를 짚지어 둔다. 검증 지점 — 파이프라인 단계 사이, 블랙보드 갱신 지점마다 잘못된 값을 걸러낼 감시 장치를 둔다.

이런 운영 이슈는 프로토콜 계층이 대신 해결해 주지 않는다. MCP와 A2A는 메시지가 오가는 형식을 표준화할 뿐, 그 메시지가 제때 오지 않았을 때 무엇을 할지는 규정하지 않는다. 감독자·파이프라인·블랙보드 가운데 어떤 패턴을 택하든, 실패를 감지하고 되돌리고 알리는 구체적인 절차는 오케스트레이션을 설계하는 사람의 몫으로 고스란히 남는다. 패턴은 출발점일 뿐이고, 그 패턴이 실패 앞에서 어떻게 행동할지까지 정해졌을 때 비로소 조율 설계는 완성된다. 그리고 조율이 흔들리는 바로 그 순간, 각 에이전트에게 무엇까지 허용되어 있었는지가 사고의 파급 범위를 결정한다 — 다음 장이 다룰 권한의 경계 설계가 오케스트레이션의 마지막 조각인 이유다.

05

신뢰와 권한의 경계 설계

에이전트가 도구를 대신 실행하고 다른 에이전트에게 작업을 넘기기 시작하면, '누가 무엇을 할 수 있는가'의 경계를 어떻게 그을 것인가가 핵심 질문이 된다. 도구 노출·협업·지휘라는 세 지점에서 권한이 새어 나가는 경로를 하나씩 짚고, 역할별 접근 범위를 좁혀 잡는 최소 권한의 원칙을 경계 설계의 언어로 옮긴다.

에이전트가 사람을 대신해 도구를 실행하고, 나아가 다른 에이전트에게 작업 일부를 넘기기 시작하면 질문의 축이 바뀐다. "무엇을 할 수 있는가"가 아니라 "누가, 어떤 조건에서, 어디까지 할 수 있는가"가 중심에 선다. 자율성이 커질수록 이 경계선은 성기게 그어서는 안 되고 오히려 촘촘해져야 한다. 위험은 한 층위에서만 생기지 않는다. 도구를 노출하는 지점, 에이전트끼리 대화하는 지점, 여러 에이전트를 지휘하는 지점 — 세 층위마다 성격이 다른 구멍이 있다.

5.1 도구 노출 지점: MCP 서버의 과잉 권한

MCP 서버는 파일 시스템, 데이터베이스, 외부 API를 도구로 감싸 에이전트에게 넘긴다. 문제는 이 감싸기가 흔히 필요보다 넓은 범위로 이루어진다는 데 있다. 읽기만 필요한 작업에 쓰기 권한까지 딸린 도구, 특정 폴더만 다루면 되는데 드라이브 전체에 접근하는 도구가 그런 예다. 서버를 만드는 쪽은 "일단 넓게 열어 두고 나중에 좁히자"는 편을 택하기 쉬운데, 나중에 실제로 오는 경우는 드물다. 에이전트는 프롬프트에 따라 설계자가 예상하지 못한 도구 조합을 시도할 수 있으므로, 노출된 권한의 상한이 곧 실제 위험의 상한이 된다. 서버를 설계할 때는 기능 단위가 아니라 권한 단위로 도구를 쪼개는 편이 안전하다. 하나의 도구가 여러 권한을 한꺼번에 담고 있으면, 그중 하나만 필요한 호출에서도 나머지 권한까지 함께 딸려 나가기 때문이다.

5.2 협업 지점: A2A에서 상대를 검증하는 문제

A2A로 작업을 넘길 때는 상대가 자칭하는 신원과 실제로 하는 행동이 같은지 확인할 방법이 필요하다. 에이전트 카드에 적힌 역할과 실제 호출되는 기능이 어긋날 수 있고, 위임받은 에이전트가 원래 요청 범위를 넘어서는 부수 작업을 시도할 수도 있다. 사람 사이의 협업이라면 평판과 직관이 이런 간극을 메우지만, 에이전트 사이에는 그

런 완충이 없다. 카드에 서명이 있다고 해서 그 서명이 지금 이 요청의 정당성까지 보증하지는 않는다. 그래서 신원 확인과 행동 범위 확인을 분리해, 요청마다 "이 에이전트가 맞는가"와 "이 요청이 허용된 범위 안인가"를 각각 따로 판정해야 한다. 둘 중 하나만 확인하고 통과시키면, 신원은 맞지만 범위를 넘어서 요청이나 범위는 맞지만 신원이 위조된 요청을 놓치게 된다.

5.3 지휘 지점: 위임 체인 전체의 최소 권한

오케스트레이터가 여러 에이전트에게 작업을 나눠 맡기면 권한은 상위에서 하위로 흘러 내려간다. 이때 흔한 실수는 상위 에이전트의 넓은 권한을 하위 에이전트에게 그대로 물려주는 것이다. 위임은 권한의 복제가 아니라 축소여야 한다. 각 하위 작업이 실제로 필요로 하는 범위만큼만 잘라 넘기고, 체인이 몇 단계를 거치든 누적된 권한이 원래 필요보다 넓어지지 않도록 매 단계에서 다시 좁혀야 한다. 위임 체인이 길어질수록 이 원칙은 지키기 어려워진다. 세 번째, 네 번째 위임 단계에 이르면 애초에 어떤 권한이 왜 필요했는지 추적하기 힘들어지고, 그 틈에 권한은 조금씩 다시 넓어지는 방향으로 흐른다.

완화의 세 원칙

최소 권한 — 작업에 필요한 최소 범위만 부여한다. 감사 로그 — 어떤 에이전트가 무엇을 언제 실행했는지 전부 남긴다. 사람 승인 체크포인트 — 되돌리기 어렵거나 범위가 넓은 실행 앞에서는 자동화를 멈추고 사람에게 확인받는다.

세 층위의 위험은 성격이 다르지만 완화 원리는 하나로 모인다. 권한을 촘촘히 나누고, 실행을 기록하고, 결정적인 지점에서 사람을 개입시키는 것이다. 이 원칙을 구체

적인 그림으로 옮기면, 역할마다 어떤 도구와 데이터에 접근할 수 있는지를 한눈에 대조하는 표가 유용해진다.

	도구 자원		데이터 자원	
	MCP 도구	외부 API	사용자 데이터	시스템 로그
조회	조회	없음	조회	조회
실행	실행	실행	조회	조회
관리	관리	관리	관리	관리

그림 5-1 역할별 접근 권한 범위를 대조한 예시 권한 매트릭스 — 조회·실행·관리 세 역할이 도구·데이터 자원 각각에 대해 갖는 권한 수준을 표시하며, 칸의 색이 짙을수록 권한 범위가 넓다 · 출처: 본문 원칙 정리

이런 매트릭스는 한 번 그려두고 끝나는 문서가 아니다. 새 도구가 추가되거나 위임 체인에 에이전트가 하나 더 들어올 때마다 다시 검토해야 하는, 계속 살아 움직이는 기준이다. 권한의 경계는 시스템이 멈춰 있을 때 그 선이 아니라, 시스템이 계속 변할 것을 전제로 반복해서 다시 긋는 선이어야 자율성의 증가를 따라잡을 수 있다.

위임은 신뢰의 확장이 아니다

작업을 넘겼다고 그 에이전트를 무조건 신뢰해야 하는 것은 아니다. 위임 이후에도 상위 에이전트는 결과를 검증할 책임을 그대로 진다.

경계 설계 점검

도구는 기능 단위가 아니라 권한 단위로 쪼갬다. 협업 요청은 신원과 행동 범위를 각각 따로 판정한다. 위임은 권한의 복제가 아니라 축소로, 매 단계에서 범위를 다시 좁힌다.

결국 세 층위의 경계 설계는 따로 놓인 과제가 아니라 하나로 이어진 사슬이다. MCP 서버에서 도구 권한을 좁게 잘라도 A2A에서 상대를 제대로 검증하지 못하면 그 좁은 권한이 엉뚱한 손에 넘어가고, 오케스트레이션에서 위임을 축소하지 않으면 아래 단계로 갈수록 권한이 다시 부풀어 오른다. 어느 한 층위만 단단히 잠그고 나머지를 방치하면, 전체 경계의 강도는 가장 느슨한 층위 수준으로 떨어진다. 자율성을 늘리는 설계와 경계를 촘촘히 하는 설계는 같은 속도로 함께 가야 하는 두 바퀴다. 경계 설계를 뒤로 미루면 그 비용은 사라지지 않고, 시스템이 커진 만큼 불어난 채로 돌아온다.

06

표준 경쟁과 상호운용성

완전히 통일된 표준이 자리잡기 전, 조직은 지금 무엇을 준비해야 하는가.
경쟁하는 제안들이 부분 채택과 어댑터를 거쳐 상호운용 계층으로 수렴해 온
표준화의 일반적 경로를 확인하고, 그 지형 위에서 오늘 당장 취할 수 있는 실행
원칙을 추린 뒤, 다섯 장을 관통해 온 흐름을 하나로 모아 이 책을 맺는다.

지금까지 다섯 개 장에서 MCP-A2A-오케스트레이션-권한 설계를 각각 따로 살펴왔다. 이 넷은 별개의 기술이 아니라, 에이전트 시스템이 커질 때 반드시 마주치는 네 개의 질문 — 도구를 어떻게 부를지, 다른 에이전트에게 어떻게 일을 맡길지, 여러 에이전트를 어떻게 지휘할지, 누구에게 어디까지 권한을 줄지 — 에 대한 답이다. 문제는 이 답들이 아직 하나의 표준으로 완전히 통일되지 않았다는 데 있다. 여러 진영이 비슷한 문제에 각자의 제안을 내놓고 있고, 조직은 그 경쟁이 끝나기를 기다릴 여유 없이 지금 시스템을 짜야 한다.

6.1 경쟁하는 제안들이 수렴해 가는 경로

새로운 프로토콜이 자리 잡는 과정은 대체로 비슷한 경로를 밟는다. 초기에는 여러 제안이 동시에 등장해 한동안 병존한다. 각 제안은 서로 다른 조직·프레임워크에서 나왔기 때문에 메시지 형식도, 다루는 범위도 조금씩 다르다. 시간이 지나면 일부 조직이 먼저 특정 제안을 채택해 실제로 써 보고, 이 과정에서 제안 간의 격차와 빈틈이 드러난다. 여기서 흔히 나타나는 다음 단계가 게이트웨이-어댑터 같은 중간 계층이다 — 서로 다른 제안을 그대로 하나로 합치는 대신, 그 사이를 변환해 주는 얇은 층을 끼워 넣는 방식이다.

표준화의 일반적 성격

표준이 하나로 정리된다는 것이 반드시 승자 하나가 나머지를 밀어낸다는 뜻은 아니다. 흔한 경로는 경쟁하던 제안들이 상호운용 계층 아래서 공존하며, 그 계층을 거치면 어느 쪽을 쓰든 큰 차이가 없어지는 쪽으로 정리되는 것이다.

이 상호운용 계층이 두터워질수록 최종 사용자와 개발자는 어느 제안이 이겼는지를 점점 신경 쓰지 않게 된다. 표준 경쟁의 결과가 조용히 인프라 안으로 흡수되는 셈이

다. 이 책이 다룬 시점에서 특정 제안의 우열을 단정하기는 이르지만, 병존에서 상호 운용 계층을 거쳐 정리되어 가는 흐름 자체는 이전의 여러 기술 표준에서 반복적으로 관찰된 패턴이다.

이 경로에서 눈여겨볼 지점은, 상호운용 계층이 등장한다고 해서 원래의 경쟁이 곧바로 무의미해지지는 않는다는 것이다. 게이트웨이나 어댑터는 서로 다른 메시지 형식을 맞춰 줄 뿐, 각 제안이 원래 가정한 능력의 범위나 신뢰 모델까지 완전히 지워 주지는 않는다. 그래서 초기 국면에는 변환 계층 하나를 더 얹는 비용과, 처음부터 한 제안에 맞춰 짜는 단순함 사이에서 매번 판단이 필요하다. 이 판단을 프로젝트마다 새로 하지 않도록, 조직 차원의 원칙을 세워 두는 편이 낫다.

6.2 지금 조직이 취할 수 있는 실행 원칙

표준이 하나로 정리되기를 기다리는 동안에도 시스템은 지어야 한다. 여기서 몇 가지 원칙이 실무적인 완충 역할을 한다. 첫째, 특정 프로토콜의 세부 사양에 시스템 전체를 강하게 결합하지 않는 아키텍처를 짠다 — 프로토콜이 말하는 부분을 교체 가능한 경계로 감싸 두면, 사양이 바뀌거나 상호운용 계층이 등장했을 때 시스템 전체를 다시 짤 필요가 없다. 둘째, 특정 벤더의 구현에만 의존하는 종속을 최소화한다. 셋째, 거버넌스는 시스템이 커진 뒤 덧붙이는 것이 아니라 처음 설계 단계부터 함께 넣는다 — 5장에서 다룬 최소 권한과 감사 로그는 나중에 끼워 넣기 어렵다. 넷째, 표준 동향을 한번 확인하고 끝내지 않고 주기적으로 재점검하는 절차를 팀 안에 둔다.

6.3 다섯 장을 관통한 흐름

MCP는 에이전트와 도구 사이의 수직 연결을 표준화해, 도구마다 커스텀 통합을 반복하던 불편화를 줄였다. A2A는 그 위에서 에이전트와 에이전트를 잇는 수평 연결을

다뤄, 서로 다른 조직이 만든 에이전트가 내부 구현을 몰라도 협업할 수 있게 했다. 오케스트레이션은 이 두 연결이 표준화된 뒤에도 여전히 개발자의 몫으로 남는 지휘의 문제 — 여러 에이전트를 하나의 목표로 조율하는 패턴 — 를 다뤘다. 마지막으로 권한 설계는 이 모든 연결과 위임이 늘어날수록 함께 촘촘해져야 하는 경계선을 세 층위로 나눠 짚었다.

이 넷을 나란히 놓고 보면 공통된 축이 하나 드러난다. 아래 층위(MCP)에서 위 층위(권한 설계)로 갈수록 표준화의 정도는 벌어지고, 조직이 스스로 판단해야 할 몫은 커진다는 것이다. 프로토콜이 메시지 형식을 정해 줄 수는 있어도, 그 프로토콜을 어떤 경계 안에서 쓸지, 누구에게 얼마만큼의 권한을 내줄지는 결국 그 시스템을 짓는 조직의 설계 판단으로 남는다. 표준 경쟁이 정리되어도 이 판단의 몫은 사라지지 않는다.



그림 6-1 경쟁하는 제안들이 상호운용 계층으로 수렴해 가는 표준화의 일반적 단계 · 출처: 저자 정리

프로토콜은 계속 바뀌겠지만, 도구를 안전하게 연결하고 위임을 투명하게 관리하며 권한을 촘촘히 설계해야 한다는 문제 자체는 사라지지 않는다. 지금 그 원칙을 아키텍처에 새겨 두는 조직이, 표준이 정리된 다음에도 시스템을 다시 짜지 않아도 되는 조직이다.

지금 점검할 세 가지 질문

프로토콜 하나가 바뀌면 시스템의 어느 부분까지 함께 고쳐야 하는가. 지금 쓰는 구현을 다른 것으로 바꾸려면 무엇이 발목을 잡는가. 권한과 감사 로그는 지금 설계에 이미 들어 있는가, 아니면 나중에 미뤄 둔 항목인가 — 이 세 질문에 바로 답할 수 없다면, 표준이 정리되기 전에 먼저 점검할 지점이 있다는 뜻이다.

